

Unidad Didáctica

**Estructuras de Control,
Unidad III**

Contenido

Introducción	3
Desarrollo de Contenidos	4
Estructuras de control	4
Estructura algorítmica Secuencial.....	4
Estructuras algorítmicas Selectivas.....	5
Estructuras algorítmicas Repetitivas	7
Asignaciones especiales	7
Introducción.....	9
Bibliografía y Webgrafía	12
Glosario	13

Introducción



base a ciertas condiciones. En este sentido, podemos hablar de tres tipos de estructuras: la secuencial, la selectiva y la repetitiva.

¡Bienvenido! En esta ocasión hablaremos sobre estructuras de control, un tema fundamental en programación. Las estructuras de control son aquellas que nos permiten controlar el flujo de un programa y tomar decisiones en

La estructura secuencial se refiere a la ejecución de las instrucciones en un orden determinado, es decir, una tras otra. En cambio, la estructura selectiva nos permite elegir qué instrucciones ejecutar en función de una condición. Dentro de las estructuras selectivas encontramos la alternativa simple, la doble y la múltiple, cada una con sus particularidades y aplicaciones.

Por último, las estructuras repetitivas nos permiten repetir un conjunto de instrucciones varias veces en función de una condición o un contador. En este grupo encontramos la estructura for, while-do y do-while, cada una con sus ventajas y desventajas. También hablaremos sobre las asignaciones especiales como el contador y el acumulador, y cómo utilizarlos en nuestras estructuras repetitivas.

En esta introducción también hablaremos sobre el formato y el funcionamiento de cada una de estas estructuras, y cómo utilizarlas de manera eficiente en nuestros programas. Además, veremos algunos ejemplos prácticos de cómo aplicar estas estructuras en diferentes situaciones.

Esperamos que esta introducción te motive a profundizar en el mundo de la programación y a entender la importancia de las estructuras de control. ¡Comencemos!

Desarrollo de Contenidos

Estructuras de control

Estructura algorítmica Secuencial

La estructura algorítmica secuencial es la más simple de todas y se refiere a la ejecución de una secuencia de instrucciones en un orden determinado. Es decir, cada instrucción se ejecuta una detrás de otra en el orden en que aparecen en el algoritmo.

En Python, la estructura algorítmica secuencial se representa simplemente mediante la secuencia de instrucciones que se escriben en un programa, una detrás de otra. Por ejemplo, si queremos imprimir en pantalla un mensaje seguido de otro, simplemente escribimos dos instrucciones de impresión (print) en orden:

```
print("Hola")  
print("Mundo")
```

La salida en la consola sería:

```
Hola  
Mundo
```

Otro ejemplo de estructura algorítmica secuencial en Python podría ser el siguiente programa que solicita al usuario que ingrese dos números y luego calcula y muestra en pantalla su suma:

```
numero1 = int(input("Ingrese el primer número: "))  
numero2 = int(input("Ingrese el segundo número: "))  
suma = numero1 + numero2  
print("La suma de los dos números es:", suma)
```

En este ejemplo, las instrucciones se ejecutan secuencialmente en el orden en que aparecen en el programa. Primero, el usuario ingresa dos números, luego se calcula la suma y finalmente se muestra en pantalla el resultado.

Es importante tener en cuenta que la estructura algorítmica secuencial es la base para construir algoritmos más complejos que involucran otras estructuras algorítmicas, como la estructura algorítmica selectiva o la estructura algorítmica repetitiva.

Estructuras algorítmicas Selectivas

Introducción

Las estructuras algorítmicas selectivas en Python son aquellas que permiten que una acción se ejecute solo si se cumple una condición específica. Estas estructuras utilizan operadores de comparación, como igualdad (==), desigualdad (!=), menor que (<), mayor que (>), entre otros. En Python, las estructuras selectivas más comunes son la instrucción "if" y la instrucción "if-else". También es posible utilizar una estructura selectiva anidada "if-elif-else" para establecer múltiples condiciones y acciones asociadas.

Alternativa simple (if-then)

La alternativa simple (if-then) es una estructura de control selectiva que permite ejecutar una acción si se cumple una condición específica. En Python, la sintaxis básica de la estructura if-then es la siguiente:

```
if condicion:  
    # código que se ejecutará si se cumple la condición
```

Donde "condicion" es una expresión booleana que se evalúa como verdadera o falsa. Si la condición es verdadera, se ejecutará el bloque de código indentado debajo del if. Si la condición es falsa, el bloque de código será ignorado.

Aquí hay un ejemplo simple que utiliza la estructura if-then para imprimir un mensaje si una variable llamada "x" es mayor que 5:

```
x = 7  
if x > 5:  
    print("x es mayor que 5")
```

En este ejemplo, la condición "x > 5" se evalúa como verdadera, por lo que se imprime el mensaje "x es mayor que 5". Si la variable "x" tuviera un valor menor o igual a 5, el mensaje no se imprimiría.

Alternativa doble (if-then-else)

También es posible agregar una cláusula else a la estructura if-then para especificar un bloque de código que se ejecutará si la condición es falsa. La sintaxis básica para la estructura if-then-else en Python es la siguiente:

```
if condicion:
    # código que se ejecutará si se cumple la condición
else:
    # código que se ejecutará si no se cumple la condición
```

Aquí hay un ejemplo que utiliza la estructura if-then-else para imprimir un mensaje diferente

si la variable "x" es mayor que 5 o no:

```
x = 3
if x > 5:
    print("x es mayor que 5")
else:
    print("x es menor o igual a 5")
```

En este ejemplo, la condición "x > 5" se evalúa como falsa, por lo que se ejecuta el bloque de código después de la cláusula else, y se imprime el mensaje "x es menor o igual a 5".

Estructura selectiva en cascada (anidadas)

La estructura de control alternativa múltiple, también conocida como "if-elif-else", permite evaluar múltiples condiciones y ejecutar diferentes bloques de código según cada una de ellas. En Python, se utiliza la sintaxis if-elif-else para esta estructura.

La sintaxis básica es la siguiente:

```
if condicion1:
    # bloque de código si se cumple la condición1
elif condicion2:
    # bloque de código si se cumple la condición2
elif condicion3:
    # bloque de código si se cumple la condición3
else:
    # bloque de código si no se cumple ninguna de las condiciones anteriores
```

Cada condición se evalúa en orden, y el bloque de código correspondiente al primer condicional que se cumpla será ejecutado. Si ninguna de las condiciones se cumple, se ejecutará el bloque de código dentro del "else".

Aquí hay un ejemplo sencillo en Python que utiliza la estructura if-elif-else:

```
edad = 18

if edad < 18:
    print("Eres menor de edad")
elif edad >= 18 and edad < 30:
    print("Eres joven")
elif edad >= 30 and edad < 60:
    print("Eres adulto")
else:
    print("Eres mayor de edad")
```

En este ejemplo, se evalúan varias condiciones basadas en la edad de una persona. El primer bloque de código se ejecutará si la edad es menor que 18, el segundo bloque de código si la edad está entre 18 y 30 años, el tercer bloque de código si la edad está entre 30 y 60 años, y el último bloque de código si la edad es mayor o igual a 60 años.

Cada bloque de código puede contener múltiples instrucciones de Python y, en general, se puede utilizar cualquier tipo de condición que produzca un valor booleano (verdadero o falso).

Estructuras algorítmicas Repetitivas

Asignaciones especiales

Contador

Un contador es una variable que se utiliza para realizar un seguimiento del número de veces que ocurre un evento o para llevar un registro de una cantidad. En Python, se puede crear un contador utilizando una variable y luego incrementando su valor en un bucle o cuando se cumpla una determinada condición.

Aquí hay un ejemplo de cómo crear un contador en Python:

```
contador = 0

for i in range(1, 11):
    if i % 2 == 0:
        contador += 1

print("El número de números pares en el rango es:", contador)
```

En este ejemplo, creamos un contador llamado contador e inicializamos su valor en cero. Luego, usamos un bucle for para recorrer los números del 1 al 10. Si el número actual es par, aumentamos el valor del contador en 1. Al final del bucle, imprimimos el valor del contador, que representa la cantidad de números pares en el rango.

También se puede utilizar un contador para realizar un seguimiento de una cantidad, como el número de veces que se ha realizado una acción. Por ejemplo:

```
contador = 0

while True:
    respuesta = input("¿Quieres continuar? (S/N): ")
    if respuesta.upper() == "S":
        contador += 1
    else:
        break

print("Has presionado 'S' %d veces." % contador)
```

En este ejemplo, creamos un contador llamado contador e inicializamos su valor en cero. Luego, utilizamos un bucle while para solicitar al usuario que ingrese una respuesta 'S' o 'N'. Si el usuario ingresa 'S', incrementamos el valor del contador en 1. Si el usuario ingresa 'N', rompemos el bucle. Al final, imprimimos el valor del contador, que representa el número de veces que el usuario ingresó 'S'.

Acumulador

El acumulador es una variable que se utiliza para almacenar la suma de valores en un bucle o ciclo. Cada vez que se recorre el bucle, se agrega el valor actual a la variable acumuladora. Al final del bucle, la variable acumuladora contiene la suma total de los valores.

Aquí hay un ejemplo en Python que utiliza un acumulador para sumar los valores de una lista:

```
numeros = [2, 5, 7, 1, 3]
acumulador = 0

for numero in numeros:
    acumulador += numero

print("La suma de los numeros es:", acumulador)
```

En este ejemplo, se define una lista llamada "numeros" que contiene algunos valores enteros. A continuación, se define una variable acumuladora "acumulador" inicializada

en cero. Luego, se utiliza un ciclo for para recorrer todos los elementos de la lista "numeros". Dentro del ciclo, se agrega el valor actual a la variable acumuladora usando el operador +=. Al final del ciclo, la variable "acumulador" contiene la suma total de los valores de la lista. Finalmente, se imprime el resultado en la consola.

Introducción.

Las estructuras repetitivas son un elemento fundamental en la programación, y en particular en Python, ya que permiten ejecutar un bloque de código varias veces. La capacidad de repetir una tarea de manera automatizada es esencial para cualquier aplicación, ya que se pueden procesar grandes cantidades de datos o realizar cálculos complejos de manera eficiente. Además, estas estructuras permiten una mayor flexibilidad en la programación, ya que el número de repeticiones puede variar en función de diferentes condiciones.

En Python, existen dos estructuras repetitivas principales: while y for. La estructura while se utiliza para repetir un bloque de código mientras se cumpla una condición determinada, mientras que la estructura for se utiliza para iterar sobre una secuencia, como una lista o un rango de números. Ambas estructuras son muy útiles en diferentes situaciones, desde procesamiento de datos hasta la creación de algoritmos complejos. Por lo tanto, es fundamental tener un buen conocimiento de estas estructuras para poder crear programas efectivos y eficientes.

Estructura repetitiva for.

En Python, la estructura repetitiva "for" se utiliza para iterar sobre una secuencia (una lista, una tupla, un diccionario, un conjunto, etc.) y realizar una acción para cada elemento de la secuencia. Esta estructura se compone de tres elementos principales: la palabra clave "for", una variable que representa cada elemento de la secuencia y la propia secuencia.

El siguiente es un ejemplo básico de la estructura repetitiva "for" en Python, donde se itera sobre una lista de números y se imprime cada número en la consola:

```
numeros = [1, 2, 3, 4, 5]
for num in numeros:
    print(num)
```

En este ejemplo, "numeros" es la secuencia que se va a iterar y "num" es la variable que representa cada número de la lista en cada iteración del ciclo "for". La función "print()" se utiliza para imprimir cada número en la consola. En cada iteración del ciclo "for", "num" toma el valor del siguiente número de la lista, empezando por el primer elemento (1) y terminando en el último elemento (5). Por lo tanto, el resultado en la consola sería:

```
1
2
3
4
5
```

Estructura repetitiva while-do.

La estructura repetitiva while-do es una herramienta fundamental en la programación que permite ejecutar una serie de instrucciones de forma repetida mientras se cumpla una determinada condición. En otras palabras, se utiliza cuando se necesita realizar una acción de forma repetitiva hasta que se alcance cierta condición. La sintaxis básica de esta estructura es la siguiente:

```
while condicion:
    # instrucciones a ejecutar mientras se cumple la condición
```

El ciclo se ejecutará siempre y cuando la condición sea verdadera. Una vez que la condición se vuelva falsa, la ejecución del programa continuará con la siguiente instrucción después del bloque while. Aquí un ejemplo sencillo que utiliza la estructura repetitiva while-do para imprimir los números del 1 al 5:

```
num = 1
while num <= 5:
    print(num)
    num += 1
```

Este código define una variable num y utiliza la estructura while para imprimir los valores de num del 1 al 5. La condición del while es que num sea menor o igual a 5, y dentro del ciclo se imprime el valor actual de num y se incrementa en 1. Cuando num alcanza el valor de 6, la condición del while ya no se cumple y el ciclo termina.

Estructuras repetitivas anidadas

Las estructuras repetitivas anidadas permiten la ejecución de una o varias estructuras de control repetitivas dentro de otra estructura repetitiva. Es decir, se pueden repetir bloques de código varias veces dentro de otro bloque de código que también se está repitiendo. Esto puede ser útil en situaciones en las que se requiere la manipulación de datos en múltiples niveles o cuando se necesitan ejecutar ciertas acciones en secuencia.

En Python, se pueden anidar bucles for o while utilizando la indentación adecuada. Por ejemplo, supongamos que queremos imprimir todas las combinaciones posibles de los números del 1 al 3 en pares ordenados. Podemos lograr esto con dos bucles for anidados de la siguiente manera:

```
for i in range(1, 4):
    for j in range(1, 4):
        print((i, j))
```

En este ejemplo, el primer bucle for recorre los números del 1 al 3, mientras que el segundo bucle for también recorre los números del 1 al 3. Dentro del segundo bucle for, se imprime una tupla que contiene los valores de i y j. El resultado de este código sería la impresión de los siguientes pares ordenados:

```
(1, 1)
(1, 2)
(1, 3)
(2, 1)
(2, 2)
(2, 3)
(3, 1)
(3, 2)
(3, 3)
```

Bibliografía y Webgrafía

Gómez, A. (2015). Fundamentos de programación: algoritmos, estructuras de datos y objetos. Pearson Educación.

Hernández, M. (2018). Aprende Python en un fin de semana. CreateSpace Independent Publishing Platform.

Técnicas de programación: estructuras algorítmicas repetitivas en C, C++, Java y Python / Jesús Carretero Pérez, Juan Carlos Moure Fernández. (2018). Madrid, España: Paraninfo.

Zabalza, V. (2019). Aprende a Programar con Python 3. Independently published.

Bell, A. B. (2016). Introducción a la programación con Python. Reverte.

Severance, C. (2016). Python para todos. CreateSpace Independent Publishing Platform.

Glosario

Acumulador: Una variable que se utiliza para almacenar la suma de valores en un bucle o ciclo. Cada vez que se recorre el bucle, se agrega el valor actual a la variable acumuladora.

Bucle for: Una estructura de control de flujo en Python que se utiliza para iterar sobre una secuencia, como una lista o un rango de números. Se compone de la palabra clave "for", una variable que representa cada elemento de la secuencia y la propia secuencia.

Bucle while: Una estructura de control de flujo en Python que se utiliza para repetir un bloque de código mientras se cumpla una condición determinada. Se compone de la palabra clave "while" seguida de una condición y un bloque de código.

Bucles for o while: Son estructuras de control repetitivas que permiten ejecutar un bloque de código varias veces mientras se cumpla una condición determinada.

Ciclo: Es una estructura de control repetitiva que ejecuta un conjunto de instrucciones varias veces mientras se cumpla una condición determinada.

Condición: Es una expresión lógica que se evalúa en verdadero o falso. En el contexto de las estructuras de control repetitivas, se utiliza para determinar si se debe seguir ejecutando el ciclo.

Contador: Una variable que se utiliza para llevar un registro del número de veces que ocurre una acción o un evento en un programa. Se incrementa o decrementa en función de ciertas condiciones.

Control de flujo: El proceso de dirigir el flujo de ejecución de un programa. Las estructuras de control de flujo, como las estructuras repetitivas, permiten al programador tomar decisiones en función de ciertas condiciones y realizar diferentes acciones en función de ellas.

Estructura algorítmica Secuencial: Es la estructura más simple que se refiere a la ejecución de una secuencia de instrucciones en un orden determinado.

Estructura repetitiva For: Es una estructura de control que permite iterar sobre una secuencia de elementos y ejecutar un bloque de código para cada elemento de la secuencia.

Estructura repetitiva While: Es una estructura de control que permite repetir un bloque de código mientras se cumpla una condición específica.

repetitivas principales son el bucle for y el bucle while.

Estructura selectiva doble (if-then-else): Es posible agregar una cláusula else a la estructura if-then para especificar un bloque de código que se ejecutará si la condición es falsa.

Estructura selectiva en cascada (anidadas): Permite evaluar múltiples condiciones y ejecutar diferentes bloques de código según cada una de ellas.

Estructura selectiva simple (if-then): Es una estructura de control selectiva que permite ejecutar una acción si se cumple una condición específica.

Estructuras algorítmicas Repetitivas: Son estructuras que permiten ejecutar repetidamente un conjunto de instrucciones.

Estructuras algorítmicas Selectivas: Son aquellas que permiten que una acción se ejecute solo si se cumple una condición específica. Las estructuras selectivas más comunes en Python son la instrucción "if" y la instrucción "if-else".

Estructuras de control: Son elementos de programación que permiten modificar el flujo de ejecución de un programa.

Estructuras repetitivas anidadas: Son estructuras de control que permiten la ejecución de una o varias estructuras repetitivas dentro de otra estructura repetitiva.

Falso: Es el valor lógico que representa que una condición es falsa o no se cumple.

Incrementar: Es aumentar en una unidad el valor de una variable o expresión.

Indentación: Es el uso de espacios o tabulaciones al principio de una línea de código para indicar la estructura del programa y definir bloques de código anidados.

Instrucción: Es una acción que realiza el programa, como una operación matemática, una asignación de valor a una variable o una llamada a una función.

Iteración: El proceso de repetir un bloque de código varias veces en un programa. Se utiliza en combinación con bucles para procesar grandes cantidades de datos o realizar cálculos complejos de manera eficiente.

Iteración: Es el proceso de repetir una secuencia de instrucciones varias veces.

Programación eficiente: El proceso de escribir código que utiliza estructuras repetitivas y otros recursos de manera óptima para maximizar la eficiencia y el rendimiento del programa.

Secuencia: Un conjunto ordenado de elementos en Python, como una lista, una tupla, un diccionario o un conjunto. Se utiliza en combinación con bucles for para iterar sobre cada elemento de la secuencia.

Tupla: Es un tipo de datos en Python que permite almacenar varios valores inmutables en una sola variable. En el ejemplo dado, se utiliza para almacenar los pares ordenados de números.

Verdadero: Es el valor lógico que representa que una condición es cierta o se cumple.